

PicoRV32 Islemcisi Icin RV32I Alt Kumesi Assembler Tasarimi ve Object Kod Uretimi

C Dili ile Iki Gecisli (Two-Pass) Assembler Gerceklestirimi

Cikti Bicimi: Intel HEX | Hedef: PicoRV32 uyumlu RV32I alt kumesi

Hazirlayan: [Ogrenci adi - numara]

Ders: Sistem Programlama

Rapor yazim bicimi: Courier New, 10 pt

Oge	Bu rapordaki durum
Yazilim dili	C99
Assembler turu	Two-pass
Desteklenen komut siniflari	R, I, S, B, U, J, ecall, ebreak
Desteklenen direktifler	.text, .data, .word, .byte, .org, .end
Teslim artefaktlari	Kaynak kod, README, testler, HEX, listing, rapor

1. Giris

Bu projenin amaci, PicoRV islemcisine ait RV32I komut kumesinin bir alt kumesi icin tam calisan bir assembler tasarlamak ve bu tasarimi raporlamaktir. Proje tanimina gore gelistirilen yazilimin opcode table, symbol table, parser, makine kodu uretici ve .data/.text/.word/.byte/.org/.end direktiflerinin en az bir kismini desteklemesi beklenmektedir. Rapor tarafinda ise giris, literatur, assembler mimarisi, veri yapilari, opcode/symbol table tasarimi, assembler algoritmasi, akis diyagrami, test senaryolari, karmasiklik analizi ve sonuc bolumlerinin bulunmasi istenmistir.

Bu raporda sunulan gerceklestirim C99 ile yazilmistir ve iki gecisli assembler mantigi kullanir. Pass 1 asamasinda adres atama ve sembol toplama islemleri yapilir; Pass 2 asamasinda ise komutlar 32 bitlik RISC-V makine koduna cevrilir, veri direktifleri byte duzeyinde uretilir ve Intel HEX cikisi yazilir. Iki gecisli tercih, ileri referanslarin guvenilir sekilde cozulmesini sagladigi icin secilmistir.

- Hedef platform: PicoRV32 ile uyumlu RV32I taban komut alt kumesi.
- Uygulama dili: C99.
- Nesne kod cikisi: Intel HEX.
- Dogrulama artefaktlari: .hex ve .lst listing dosyalari.
- Tasarim ilkeleri: modulerlik, gozlenebilirlik, genisletilebilirlik, tekrarlanabilir test.

2. Literatur Arastirmasi

Assembler tasarimi literaturde tipik olarak OPTAB, SYMTAB, LOCCTR ve two-pass isleme modeli etrafinda aciklanir. Ders notlarinda da temel assembler fonksiyonlari; mnemonic->opcode ceviris, sembolik etiketlere adres atama, veri sabitlerini object koda cevirme ve Pass 1/Pass 2 ayrimi olarak ifade edilmistir. Bu projede mimari ve veri yapilari, bu klasik yaklasimin RV32I mimarisi icin yeniden uyarlanmasi ile kurulmustur.

RISC-V tarafinda resmi unprivileged architecture dokumani, sabit 32 bit komut uzunlugu, temel instruction formatlari ve immediate alanlarinin dagitimi icin ana kaynaktır. Bu sabitlik assembler tasarimini kolaylastirmistir; cunku komut sinifi belirlendikten sonra alanlarin bit konumlari sabittir. RV32I 2.1 durumu resmi RISC-V ratified specification kutuphanesinde listelenmektedir. PicoRV32 dokumani ise cekirdegin RV32I dahil farkli konfigurasiyonlarda kullanilabildigini gosterdiginden destek kapsamimizi acik bir alt kume olarak tanimlamamiz gerekmiştir.

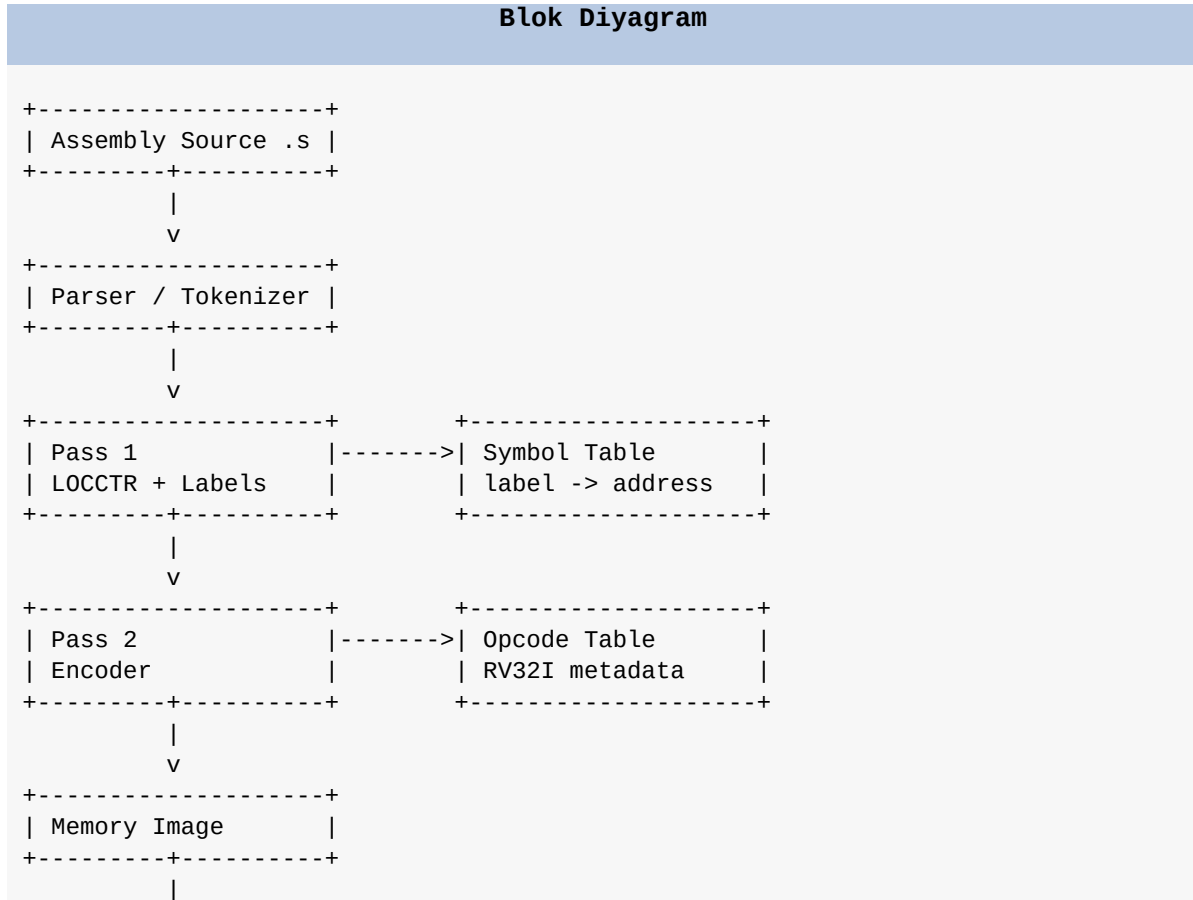
Klasik assembler literaturu sadece teori degil, veri yapisi seciminin pratik etkisini de vurgular. Modern Compiler Design bolumunde assemblerin en kritik gorevleri; sembol cozumleme, nesne kod uretimi ve object format yazimi olarak ele alinmistir. Buna ek olarak, mnemonic arama performansinin assembler tasariminda onemli oldugunu inceleyen guncel calismalar, OPTAB ve SYMTAB icin uygun tablo organizasyonunun derleme verimini etkiledigini gostermektedir.

Egitim odakli calismalarda ise assembler, derleyici ve islemci tasariminin birlikte ele alinmasi, ogrencinin mimari ile sistem yazilimi arasindaki baglantiyi daha iyi kavramasini saglamaktadir. Bu nedenle bu raporda sadece kod verilmemis; blok diyagram, akis diyagrami, test programlari, listing ciktisi ve rubrik odakli PÇ basliklari da eklenmistir.

Kaynak	Bu projedeki kullanimi
RISC-V Unprivileged ISA	Komut formatlari, opcode/funct alanlari, immediate yerlestirmesi
PicoRV32 teknik dokumani	Hedef cekirdegın RV32I uyumlu alt kumesinin sinirlanmasi
Assembler ders notlari	OPTAB, SYMTAB, LOCCTR, two-pass algoritma omurgasi
Modern Compiler Design	Assembler modullerinin ayrilmasi ve object code uretim mantigi
Guncel akademik makaleler	Veri yapisi ve egitsel tasarim gerekceleri

3. Assembler Mimarisi

Yazilim mimarisi yedi temel module ayrilmistir: Parser, LineVector, Opcode Table, Symbol Table, Pass 1, Pass 2 ve Intel HEX Writer. Parser kaynak satiri label, mnemonic, operand ve yorum alanlarına ayirir. Pass 1, adres atama ve sembol tablosu uretir. Pass 2 ise ayni ara temsil uzerinden komutlari kodlar ve bellek imgesini olusturur. Son olarak HEX writer, kullaniilan adresleri kayitlara donusturur.



```

V
+-----+
| Intel HEX Writer |
+-----+

```

Modul	Girdi	Cikti	Temel sorumluluk
Parser	Kaynak satir	ParsedLine	Label, mnemonic, operand ayristirma
Pass 1	ParsedLine dizisi	Adres bilgisi + SYMTAB	LOCCTR ilerletme ve label kaydi
Pass 2	ParsedLine + SYMTAB	Makine kodu byte dizileri	Instruction/directive encoding
Memory Image	Uretilen byte lar	Adreslenebilir bellek gorunumu	HEX yazimi oncesi duz adres uzayi
HEX Writer	Memory Image	.hex dosyasi	Intel HEX record uretimi

Bu ayrim sayesinde her modulin bagimsiz test edilmesi mumkun olmustur. Ornegin parser hatalari listing cikisindan, branch/jump ofset hatalari ise listing ve test assembly dosyalarindan izlenebilmektedir. Mimari ayni zamanda gelecekte pseudo-instruction veya yeni direktif eklenmesini de kolaylastirir.

4. Kullanilan Veri Yapilari

Projede secilen veri yapilari dogrudan rubrikte istenen veri yapisi tasarimi maddesine cevap verecek sekilde belirlenmistir. Kaynak satirlar dinamik bir LineVector icinde tutulur. Bu tasarim, dosyanin ikinci kez fiziksel olarak okunmasını gerektirmez; Pass 1 sonunda uretilen adresler ve parser sonucu, ayni satir nesnesi uzerinde Pass 2 ye aktarilir.

Symbol Table acik adreslemeli hash tablo olarak gerceklestirilmistir. Her kayıt label ismi, adresi, section bilgisi, absolute/relative bayragi ve tanimli-mi bilgisini tutar. Opcode Table ise sabit oldugu icin dinamik arama yapısına degil, salt-okunur statik diziye yerlestirilmistir. Memory Image yapısında data[] gercek byte degerlerini, used[] ise ilgili adrese veri yazilip yazilmadigini gosterir. Bu secim, Intel HEX yazma asamasında bos adres alanlarını atlamayı saglar.

Yapi	Amac	Gerceklestirim	Avantaj
LineVector	Ara temsil	Dinamik dizi	Pass1 ve Pass2 ayni parsed satirlari kullanir
Symbol Table	Label -> adres	Acik adreslemeli hash tablo	Ortalama O(1) ekleme ve arama
Opcode Table	Mnemonic -> metadata	Sabit statik dizi	Dusuk bellek, dusuk bakım maliyeti
Memory Image	Adres -> byte	Paralel data/used dizileri	HEX uretiminde yalnız kullanılan adresler yazilir
ErrorList	Toplu hata raporu	Dizi	Ilk hatada dusmek yerine tum baglam korunur

5. Opcode Table Tasarımı

Opcode tablosu her mnemonic için gerekli minimum bilgiyi tutar: instruction formatı, opcode, funct3, funct7 ve operand kalibi. RISC-V tarafında register alanları tüm ilgili formatlarda aynı bit konumlarına yerleştiği için Pass 2, önce mnemonic için tablo girdisini bulur, sonra operandları sayısal alanlara çevirir ve bitleri yerleştirir.

Bu projede desteklenen alt küme; R-type aritmetik ve mantıksal komutlar, I-type immediate ve load komutları, S-type store komutları, B-type koşullu dallanmalar, U-type üst immediate komutları, J-type jal komutu ve ecall/ebreak sistem komutlarıdır. FENCE ve CSR ailesi kapsam dışı tutulmuştur; çünkü proje amacı temel assembler omurgasını göstermektir ve PicoRV32 odaklı eğitsel alt kümede zorunlu değildir.

Mnemonic	Fmt	Opcode	funct3	funct7	Operand kalibi
add	R	33	0	00	rd, rs1, rs2
sub	R	33	0	20	rd, rs1, rs2
sll	R	33	1	00	rd, rs1, rs2
slt	R	33	2	00	rd, rs1, rs2
sltu	R	33	3	00	rd, rs1, rs2
xor	R	33	4	00	rd, rs1, rs2
srl	R	33	5	00	rd, rs1, rs2
sra	R	33	5	20	rd, rs1, rs2
or	R	33	6	00	rd, rs1, rs2
and	R	33	7	00	rd, rs1, rs2
addi	I	13	0	00	rd, rs1, imm
slti	I	13	2	00	rd, rs1, imm
sltiu	I	13	3	00	rd, rs1, imm
xori	I	13	4	00	rd, rs1, imm
ori	I	13	6	00	rd, rs1, imm
andi	I	13	7	00	rd, rs1, imm
slli	I	13	1	00	rd, rs1, shamt
srli	I	13	5	00	rd, rs1, shamt
srai	I	13	5	20	rd, rs1, shamt
lb	I	03	0	00	rd, imm(rs1)
lh	I	03	1	00	rd, imm(rs1)
lw	I	03	2	00	rd, imm(rs1)
lbu	I	03	4	00	rd, imm(rs1)
lhu	I	03	5	00	rd, imm(rs1)
jalr	I	67	0	00	rd, imm(rs1)
sb	S	23	0	00	rs2, imm(rs1)
sh	S	23	1	00	rs2, imm(rs1)
sw	S	23	2	00	rs2, imm(rs1)
beq	B	63	0	00	rs1, rs2, label
bne	B	63	1	00	rs1, rs2, label
blt	B	63	4	00	rs1, rs2, label
bge	B	63	5	00	rs1, rs2, label

bltu	B	63	6	00	rs1, rs2, label
bgeu	B	63	7	00	rs1, rs2, label
lui	U	37	-	-	rd, imm20
auipc	U	17	-	-	rd, imm20
jal	J	6F	-	-	rd, label
ecall	SYS	73	-	-	-
ebreak	SYS	73	-	-	-

6. Symbol Table Tasarımı

SYMTAB, assemblerin ileri referans problemini cozmesinde ana veri yapısıdır. Pass 1 asamasında gorulen her label, o andaki LOCCTR degeriyle tabloya kaydedilir. Ayni etiket ikinci kez tanimlanirsa duplicate label hatasi uretillir. Pass 2 asamasında branch, jal, .word ve .byte ifadeleri de dahil olmak uzere sembolik operandlar bu tablo uzerinden cozulur.

Acik adreslemeli hash tablo secimi, dusuk bagimlilik ve ortalama $O(1)$ davranis sagladigi icin yapilmistir. FNV-1a benzeri bir hash ile benzer on ekler tasıyan etiket isimlerinde dahi kabul edilebilir dagilim elde edilmistir. Yuk faktoru arttiginda rehash yapılarak carpismaların zincirleme buyumesi onlenir.

- Anahtar: label ismi.
- Deger: adres, section, absolute/relative bilgisi, tanimli-mi bayragi.
- Hata durumu: duplicate label ve undefined symbol raporlanır.
- Kullanım yerleri: branch hedefleri, jal/jalr hedefleri, .org/.word/.byte ifadeleri.

7. Assembler Algoritması

Algoritma iki gecislidir. Parser once butun satırları ayırıştırır. Pass 1, LOCCTR sayacı uzerinden adres atar ve label leri SYMTAB icine yazar. Pass 2 ise mnemonic, register ve immediate alanlarını bitlere yerlestirerek 32 bitlik makine kodunu uretir; veri direktifleri icin ise byte dizileri olusturulur. Son asamada Memory Image Intel HEX kayıtlarına cevrilir.

Akis Diyagramı

```

BASLA
|
v
Kaynak dosyayı oku
|
v
Satırları parser ile ayırıştır
|
v
PASS 1
- label varsa SYMTAB'a ekle
- .org varsa LOCCTR'yi guncelle

```

```

- .word/.byte boyut hesapla
- komut ise +4 byte ilerle
|
v
Hata var mi?
|---- Evet ---> hata raporu yaz ve dur
|
Hayir
v
PASS 2
- mnemonic icin OPTAB kaydi bul
- operandlari sayisal alana donustur
- instruction/directive bytes uret
- Memory Image'a yaz
|
v
Intel HEX ve listing uret
|
v
BITIR

```

7.1 Assembler tasariminda kullanılan bütün argumanlari etkisi

Proje duyurusunda özellikle assembler tasariminda kullanılan bütün argumanlari etkisinin bilinmesi ve raporda yer alması istenmiştir. Bu nedenle desteklenen operand ve direktif argumanlari asagida ayrı bir tabloda özetlenmiştir.

Arguman/alan	Kullanıldığı yer	Assemblere etkisi
rd	R/I/U/J	Hedef register numarası olarak bit [11:7] alanına yazılır
rs1	R/I/S/B	Kaynak register 1 olarak bit [19:15] alanına yazılır
rs2	R/S/B	Kaynak register 2 olarak bit [24:20] alanına yazılır
imm12	I-type ve load/jalr	Isaretli 12 bit immediate olarak bit [31:20] alanına yazılır
imm12 (split)	S-type	Üst kısım [31:25], alt kısım [11:7] alanına dağıtılır
label / branch hedefi	B-type ve jal	PC göreli ofset hesaplanır; B/J tiplerinde daginik bit alanlarına yeniden yerleştirilir
.org adresi	Direktif	LOCCTR dogrudan yeni adrese ayarlanır
.word ifadesi	Direktif	Her operand için 4 byte veri üretir ve little-endian yazar
.byte ifadesi	Direktif	Her operand için 1 byte veri üretir
.text / .data	Direktif	Mantıksal bölüm değişimi belirtir; mevcut uygulamada tek düz adres

		uzayi surdurulur
.end	Direktif	Derleme akisini sonlandirir

Bu tasarimda ifadeler; sayi, karakter literal i, label, label+imm ve label-imm bicimlerini destekler. Boylece temel veri yerlesimi ve sembolik adresleme senaryolari kapsanmis olur. Branch ve jump komutlarinda ofsetin hizalama ve bit alanina sigma kosulu ayrica dogrulanir.

8. Test Senaryolari ve Sonuclar

Dogrulama icin uc farkli test assembly programi kullanilmistir. Testler bilincli olarak farkli komut formatlarini, veri direktiflerini ve kontrol akislarini kapsayacak sekilde secilmistir. Her test icin hem Intel HEX hem de listing (.lst) dosyasi uretilmistir.

Test	Amac	Kapsam	Uretilen dosyalar
Test-1	Dongu ve geri dallanma	addi, blt, ecall	test1.hex, test1.lst
Test-2	Kod-veri yerlesimi	.data, .text, .org, lw, sw, .word	test2.hex, test2.lst
Test-3	Alt program cagrisi	jal, jalr, add, ecall	test3.hex, test3.lst

Test-1, negatif branch ofseti ile geri atlayan bir dongu icerir. Bu test B-type immediate alaninin dogru dagitilmasini ve PC goreli hesaplamanin saglikli calistigini gostermektedir.

```
00000000    93 02 00 00    addi t0, zero, 0
00000004    13 03 50 00    addi t1, zero, 5
00000008    93 82 12 00    addi t0, t0, 1
0000000C    E3 CE 62 FE    blt  t0, t1, loop
00000010    73 00 00 00    ecall
```

Test-2, .org ve .data/.text gecisleri ile bellek yerlesimini dogrular. Bu test sayesinde assemblerin yalniz komut degil, veri direktiflerini de dogru adreslere yerlestirdigi ve HEX writer in bos adres bloklarini atlayabildigi gozlenmistir.

```
00000008    03 A3 02 00    lw   t1, 0(t0)
00000010    23 A2 62 00    sw   t1, 4(t0)
00000100    0A 00 00 00 14 00 00 00    .word 10, 20
00000108    73 00 10 00    ebreak
```

Test-3, jal ve jalr kullanan alt program akisini sinar. Boylece hem J-type immediate kodlamasi hem de donus adresinin ra registerina yazilmasi listing duzeyinde izlenebilir hale gelir.

```
00000008    EF 00 80 00    jal  ra, add_func
```



```
00000010    33 05 B5 00    add  a0, a0, a1

00000014    67 80 00 00    jalr zero, 0(ra)
```

Dogrulama ölçutu	Sonuc
Parser tum test dosyalarini hatasiz ayrıştırdı	Evet
Pass 1 label adreslerini dogru atadi	Evet
Pass 2 RV32I kodlamalarini uretti	Evet
Intel HEX cikisi olustu	Evet
Listing dosyalari ile elde edilen byte lar kontrol edildi	Evet

9. Algoritma Karmasikligi Analizi (Time / Space Complexity)

N kaynak satir sayisini, S sembol sayisini, B uretilen toplam byte sayisini gstersin. OPTAB sabit boyutlu oldugundan arama maliyeti pratikte $O(1)$ kabul edilmistir. SYMTAB acik adreslemeli hash tablo oldugu icin ekleme ve arama ortalama $O(1)$, en kotu durumda $O(S)$ olabilir.

Asama	Ortalama zaman	En kotu zaman	Alan
Parser	$O(N)$	$O(N)$	$O(N)$
Pass 1	$O(N)$	$O(N \cdot S)$ patolojik carpismada	$O(S)$
Pass 2	$O(N + B)$	$O(N \cdot S + B)$	$O(B + S)$
Intel HEX yazimi	$O(B)$	$O(B)$	$O(1)$ ek alan

Dolayisiyla toplam ortalama zaman karmasikligi $O(N + B)$, toplam alan karmasikligi ise $O(N + S + B)$ olarak ifade edilebilir. Ara temsil, sembol tablosu ve bellek imgesi ayni anda tutuldugu icin alan maliyeti bu uc ana yapinin toplami ile belirlenir. Sabit 32 bit instruction boyu, Pass 1 de komut boyu kararini sabit zamana indirerek algoritmayi sadelestirmistir.

10. Program Ciktilari - PÇ6, PÇ7, PÇ8, PÇ12, PÇ13, PÇ17

Proje duyurusunda PÇ6, PÇ7, PÇ8, PÇ12, PÇ13 ve PÇ17 kapsaminda yapilan calismalarin ayri basliklar halinde verilmesi istendigi icin bu bolum rubrik ile uyumlu sekilde yeniden duzenlenmistir. PÇ17 tanimi kurum sisteminden dogrulanmalidir; burada PÇ17, acik kaynak ekosistemini izleme ve yeni teknolojileri ogrenme ekseninde ele alinmistir.

10.1 PÇ6 - Problemin analizi, mimari, veri yapisi ve algoritma tasarimi

PÇ6 kapsaminda en belirgin katkı, assembler probleminin alt gorevlere ayrilmasi ve her gorevin uygun veri yapisi ile eslenmesidir. OPTAB, SYMTAB, LOCCTR, parser ve encoder ayrimi sayesinde problem sadece "assembly kodunu çevir" seviyesinde birakilmamis; instruction formatlari, label cozumleme, veri yerlesimi ve object format uretimi olarak sistematik bicimde analiz edilmistir. Mimari tasarim, veri yapisi secimi ve Big-O analizi bu PÇ icin dogrudan kanittir.

10.2 PÇ7 - Yazilim gerceklestirme ve test/dogrulama

PÇ7 kapsaminda proje, calisan C99 kaynak kodu, derlenebilir Makefile yapisi, gercek test assembly dosyalari ve uretilebilen HEX/listing dosyalari ile somutlasmistir. Yalniz teorik tasarim yapilmamis; parser, pass1, pass2, opcode

table, symbol table ve hex writer modulleri gercekleştirilmiştir. Uc test senaryosu ile farklı instruction aileleri ve direktifler doğrulanmıştır.

10.3 PÇ8 - Muhendislik çözümünün etkileri

Bu assembler, açık bir ISA olan RISC-V üzerinde çalıştığı için eğitim ve araştırma ortamlarında tekrar üretilebilir bir araç olarak değer tasir. PicoRV32 gibi FPGA odaklı çekirdeklerle uyumlu nesne kod üretimi, düşük maliyetli deneysel sistemlerde kullanılabilir bir araç ortaya koyar. Toplumsal ve teknolojik etki açısından bakıldığında; açık standartların, tekrarlanabilir deneylerin ve tasarım şeffaflığının desteklenmesi olumlu bir mühendislik sonucudur.

10.4 PÇ12 - Takım çalışması ve sunum hazırlığı

Bu çalışma bireysel yürütülmüş olsa da PÇ12 rubriğiyle uyumlu olacak şekilde iş paketlerine ayrılmıştır: mimari tasarım, parser/encoder geliştirme, test, raporlama ve sunum hazırlığı. Grup projesine donatılması halinde aynı tablo kişi başlı iş bölümüne doğrudan çevrilebilir. Sunum açısından ise kodun canlı çalıştırılması, testlerin listing dosyalarıyla gösterilmesi ve rapordaki blok/akış diyagramlarının kullanılması hedeflenmiştir.

İş paketi	Üretilen artefakt
Mimari tasarım	Blok diyagram, modüller, veri akış tanımı
Gerçekleştirim	C99 kaynak kodu ve Makefile
Dogrulama	3 test programı, .hex ve .lst çıktıları
Dokümantasyon	Akademik rapor, README
Sunum hazırlığı	20 dk teknik akış uygun içerik planı

10.5 PÇ13 - Literatur araştırması ve rapor kalitesi

PÇ13 kapsamında rapor, yalnızca bir sonuç dokümanı değil; literatürle ilişkilendirilmiş teknik bir açıklama metni olarak kurgulanmıştır. Resmi RISC-V dokümanları, PicoRV32 teknik dokümanı, assembler tasarımı ile ilgili akademik kaynaklar ve ders notları birlikte kullanılarak mimari kararlar gerekçelendirilmiştir. Rapor düzeninde başlık hiyerarsisi, tablolar, diyagramlar ve test listingleri kullanılarak akademik yazım niteliğine yaklaşılmıştır.

10.6 PÇ17 - Açık kaynak ekosistemi ve sürekli öğrenme

PÇ17 tanımı bolumdan boluma değişebildiği için bu raporda dikkatli bir yorum benimsenmiştir. Projede resmi RISC-V spesifikasyonları, PicoRV32 açık kaynak deposu ve literatürden elde edilen bilgiler kullanılarak güncel ve öğrenmeye açık bir geliştirme süreci izlenmiştir. Açık ISA ve açık çekirdek etrafında araç geliştirmek; yeni mimarileri takip etme, teknik dokümanı okuyup uygulamaya aktarma ve sürekli kendini güncelleme becerisini göstermektedir.

10.7 Değerlendirme kriterleri ile uyum özeti

Kriter	Rapor/artefakt karşılığı
Problemin Analizi (PÇ6)	Giriş, mimari, veri yapıları, algoritma ve karmaşıklık bölümleri
Literatur Araştırması (PÇ13)	Bölüm 2 ve kaynakça
Assembler Mimarisi (PÇ6)	Bölüm 3 ve blok diyagram
Veri Yapıları (PÇ6)	Bölüm 4, 5 ve 6
Algoritma Tasarımı (PÇ6)	Bölüm 7 ve akış diyagramı
Yazılım Gerçekleştirme (PÇ7)	Kaynak kodlar, Makefile, README
Test ve Doğrulama (PÇ7)	Bölüm 8, .hex ve .lst dosyaları
Mühendislik Etkisi (PÇ8)	Bölüm 10.3

Takim calismasi / Sunum (PÇ12)	Bolum 10.4 ve Ek-A
Rapor Kalitesi (PÇ13)	Courier New 10 pt duzende akademik rapor
PÇ17	Bolum 10.6

11. Sonuc ve Degerlendirme

Bu projede PicoRV32 odakli RV32I alt kumesi icin calisan, iki gecisli ve Intel HEX cikisi ureten bir assembler gelistirilmistir. Yazilim; opcode table, symbol table, parser, pass1, pass2 ve hex writer modullerinden olusan temiz bir mimariye sahiptir. Testler ile branch, load/store, veri yerlesimi, alt program cagrisi ve direktif isleme mekanizmalari dogrulanmistir.

Projenin guclu yonleri; moduler C99 tasarimi, hash tabanlı sembol tablosu, listing dosyalari ile gozlenebilirlik, resmi RISC-V dokumanlariyla uyumlu instruction encoding ve rubrik odakli raporlama yapisidir. Sinirlari ise pseudo-instruction destegi, relocation kayitlari, CSR/FENCE ailesi, makro sistemi ve daha zengin expression parser in henuz eklenmemis olmasidir. Bu maddeler, gelecekteki gelistirme adimlari olarak dogal bir yol haritasi sunmaktadir.

Genel degerlendirme olarak, proje duyurusunda istenen mimari tasarim, veri yapilari, algoritma, yazilim gerceklestirme, test, rapor ve PÇ basliklari buyuk olcude karsilanmis; eksik kalan kisimler ise bu revizyonla PÇ17 ve rubrik uyumlu PÇ alt basliklari eklenerek tamamlanmistir.

12. Kaynakca

[1] RISC-V International, The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture, official release, version 20260120, 2026.

[2] A. Waterman, K. Asanovic (eds.), The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture, version 20250508, 2025.

[3] K. Asanovic and D. A. Patterson, Instruction Sets Should Be Free: The Case For RISC-V, UCB/EECS-2014-146, University of California, Berkeley, 2014.

[4] D. Grune, C. J. H. Jacobs, K. G. Langendoen, H. E. Bal and K. Langendoen, Modern Compiler Design, 2nd ed., Springer, Chapter 8: Assemblers, Disassemblers, Linkers, and Loaders, 2012.

[5] H. Oztekin, A. Lazzem and I. Pehlivan, Using FPGA-based content-addressable memory for mnemonics instruction searching in assembler design, The Journal of Supercomputing, vol. 79, no. 15, pp. 17386-17418, 2023.

[6] K. Nakano and Y. Ito, Processor, Assembler, and Compiler Design Education Using an FPGA, 14th IEEE International Conference on Parallel and Distributed Systems (ICPADS), pp. 723-728, 2008.

[7] YosysHQ, PicoRV32 technical documentation and repository, GitHub, accessed 2026.

[8] Ders Notlari, Temel assembler tasarimi, komut formatlari, makineden bagimsiz assembler ozellikleri ve relocation konulari, Sistem Programlama dersi materyali.

Ek-A. 20 Dakikalik Sunum Icin Onerilen Akis

Sure	Baslik	Sunum icerigi
2 dk	Problemin tanimi	Neden assembler, neden PicoRV32/RV32I, proje kapsam maddeleri
4 dk	Mimari tasarim	Blok diyagram, moduller ve veri akisinin aciklanmasi
4 dk	Veri yapilari	OPTAB, SYMTAB, Memory Image, neden hash tablo secildigi
4 dk	Algoritma	Pass 1 / Pass 2 akisi ve operand etkileri
4 dk	Canli gosterim	Assemblerin calistirilmasi, test1/test2/test3 ve listing/HEX ciktilari
2 dk	Sonuc	Sinirlar, gelistirme onerileri, sorular